RL IN CONTROL ENGINEERING

# Reinforcement learning for PID tuning

The third-order lag $P(s) = 1/(s+1)^3$ — from Ziegler–Nichols to a learned tuner

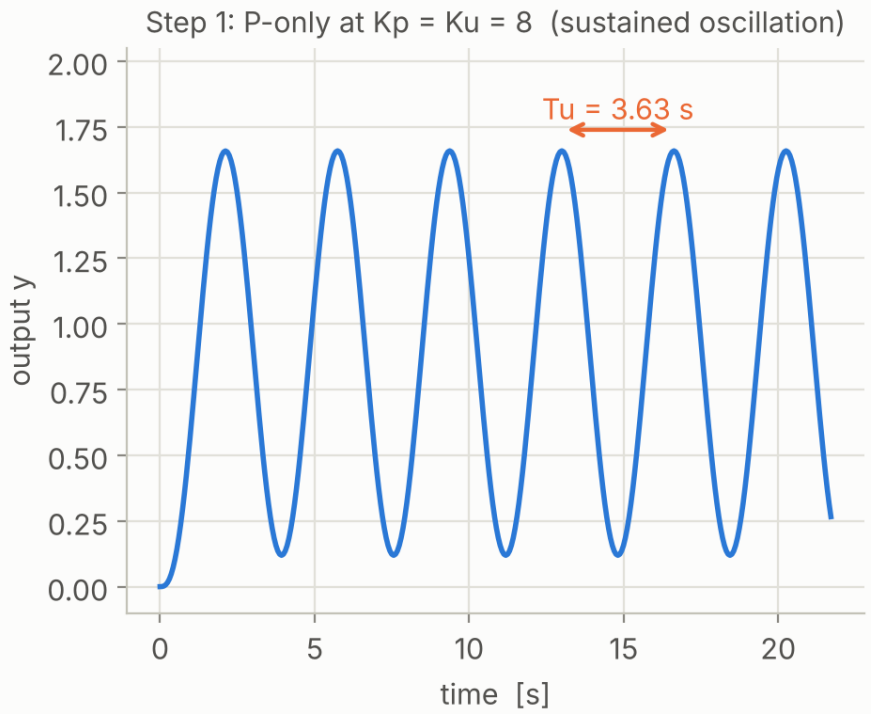
- The Ziegler–Nichols baseline and what it was designed for
- Writing the reward: tracking, load rejection, overshoot, noise
- A One plant: REINFORCE with a Gaussian policy over log-gains
- B Many plants: learning a better tuning rule
- C An iterative tuner trained with PPO
- Scorecard on unseen plants, and verifying before deployment

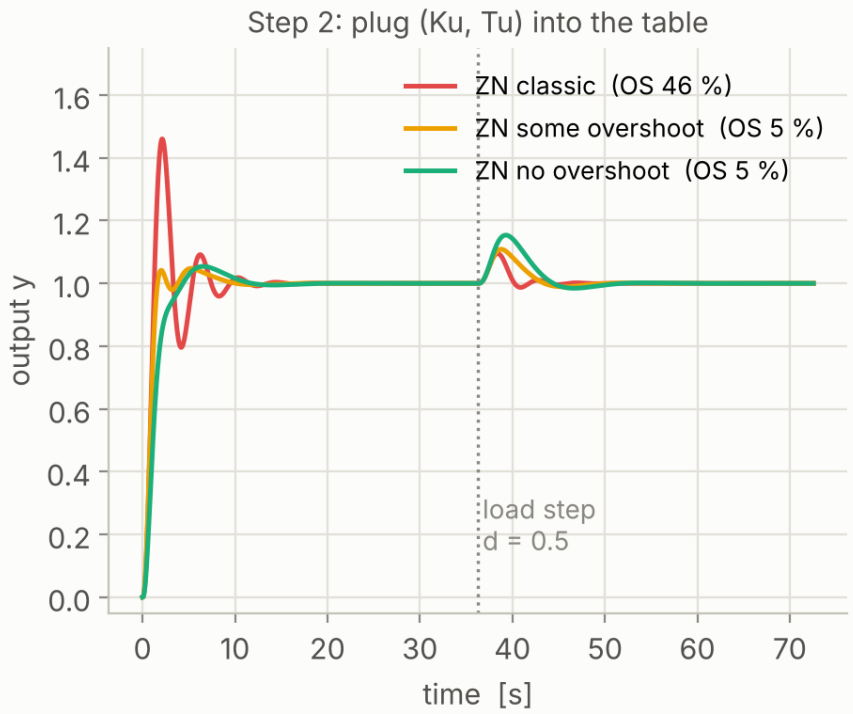
Companion notebook `RL_PID_tuning_third_order_lag.ipynb`

THE STARTING POINT

# The plant, the controller and the ZN baseline

- $P(s) = 1/(s+1)^3$ reaches −180° at $\omega = \sqrt{3}$ with gain 1/8, so $K_u = 8$ and $T_u = 2\pi/\sqrt{3} = 3.63$ s. ZN classic gives $K_p = 4.80$, $K_i = 2.65$, $K_d = 2.18$.
- $C(s) = K_p + K_i/s + K_d \cdot s$, with the derivative implemented as $K_d \cdot s/(T_f \cdot s + 1)$, $T_f = T_u/36 \approx 0.10$ s. The test is a set-point step, then a load step at the plant input.



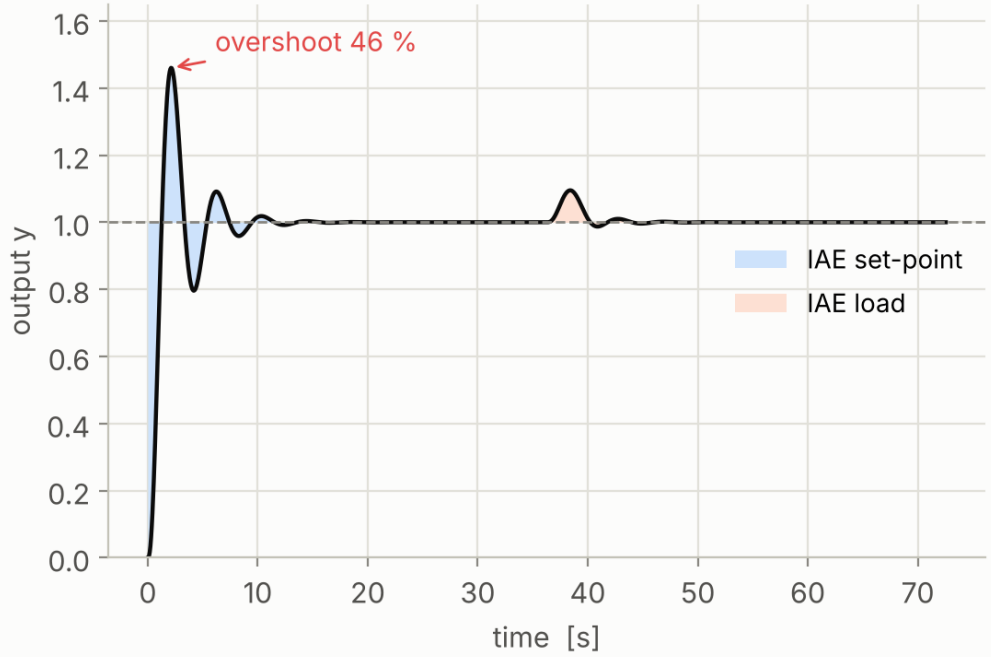



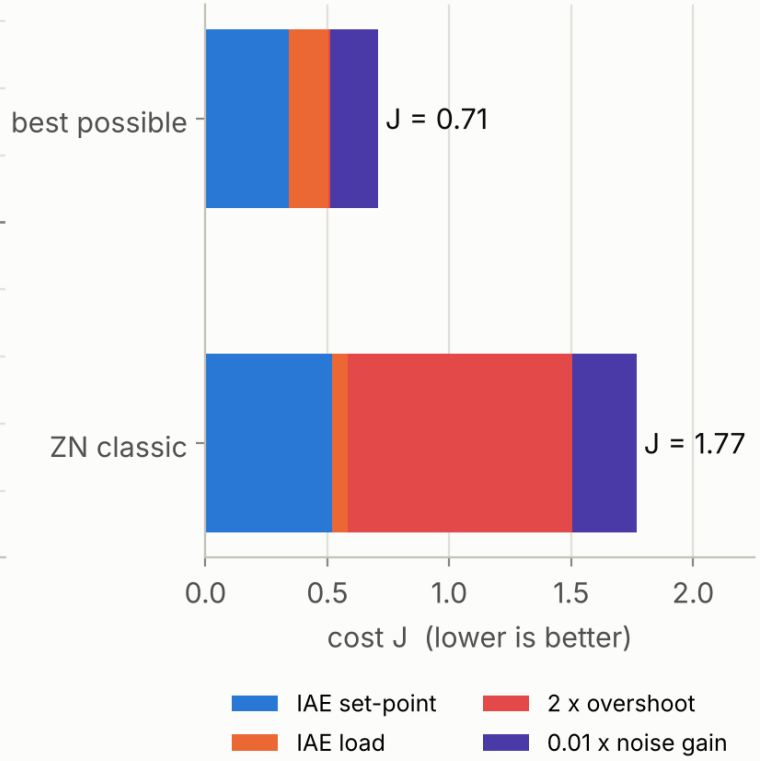

**ZN classic overshoots 46 % on the set-point step. That is not a mistake: the rule was designed for quarter-amplitude decay after a load disturbance, which it rejects quickly.**

THE COST J (REWARD = −J)

# Tuning is optimisation: first write down what good means

```
J = IAE_setpoint/Tu + IAE_load/Tu + 2·overshoot + 0.01·K(Kp + Kd/Tf)        unstable loop:  J = 20
```



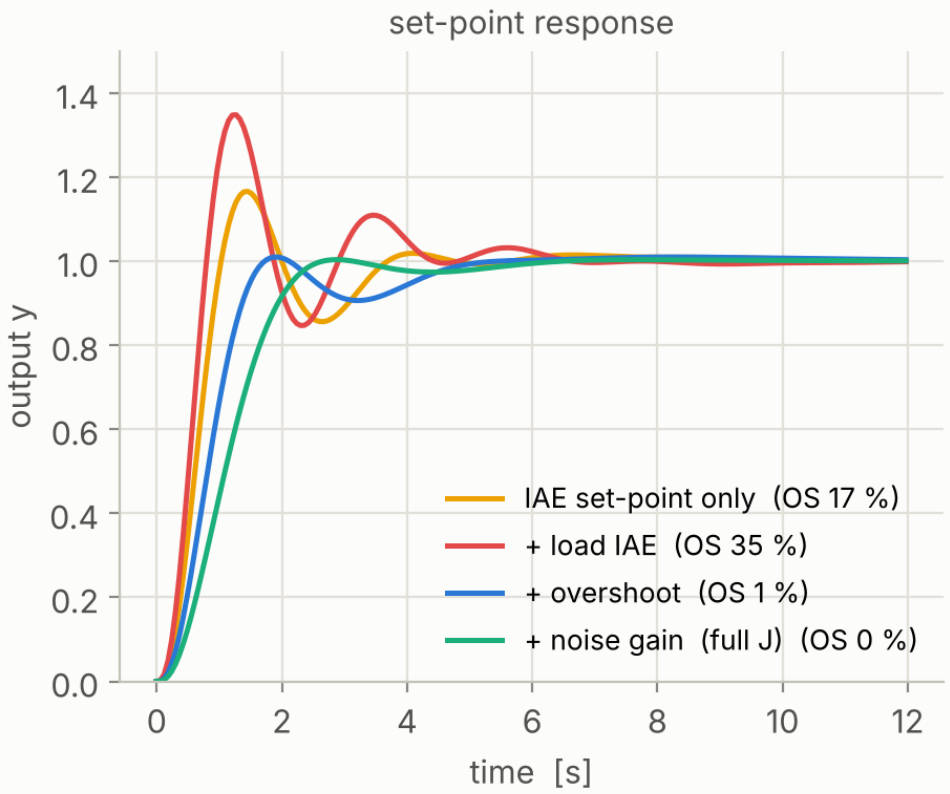



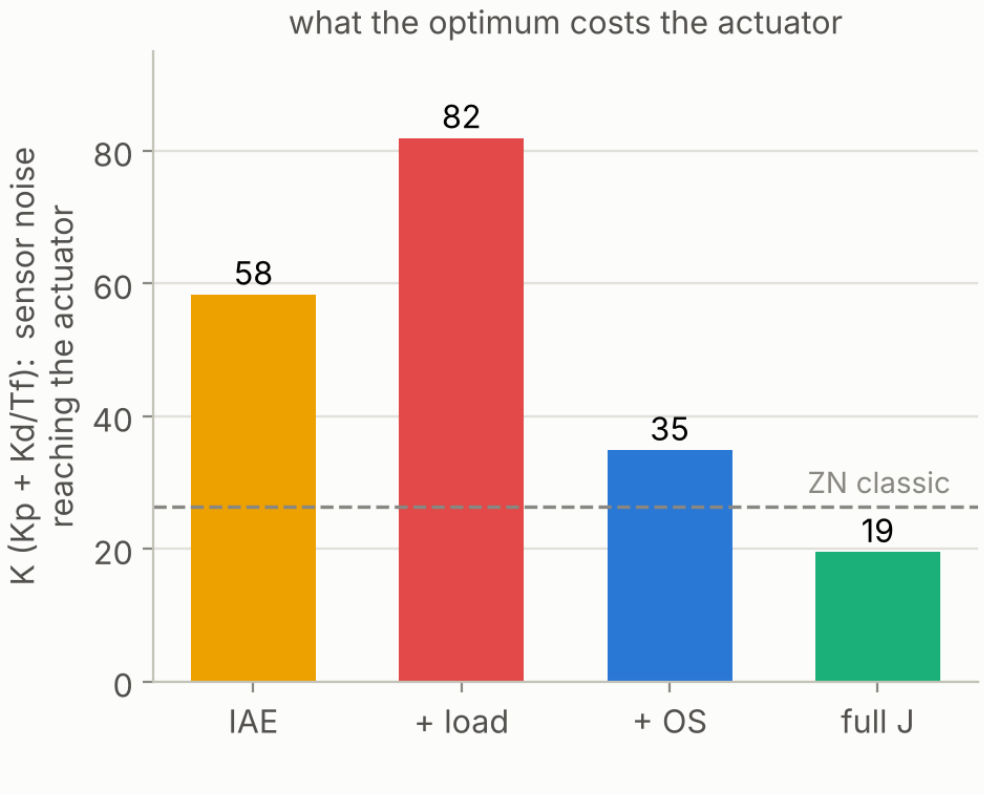

**ZN classic scores J = 1.77; the best PID for this same J scores 0.71. Most of ZN's cost is overshoot, while its load rejection is already excellent.**

REWARD DESIGN

# The agent optimises exactly what you write

- Each curve is the exact optimum of a different reward. Terms are added from left to right: set-point IAE, then load IAE, then overshoot, then noise gain.

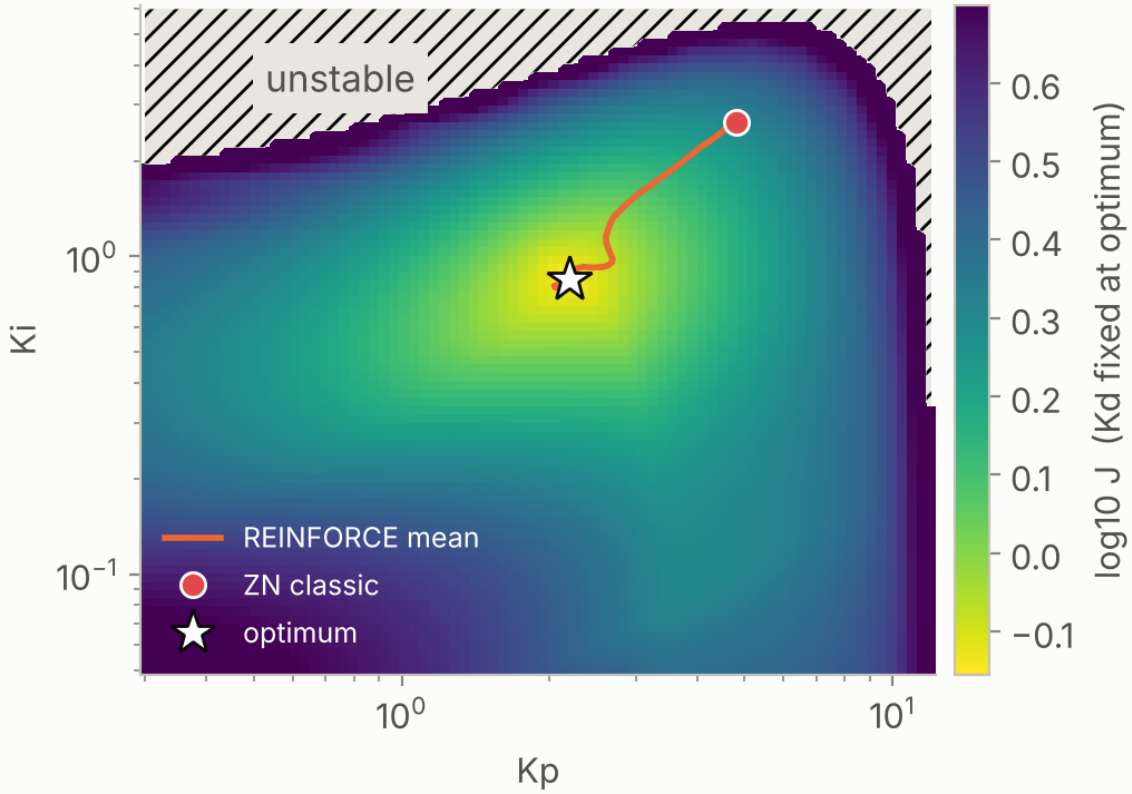



**Leave out the overshoot and noise terms and the "optimal" PID sends 4.2× more sensor noise to the actuator (82 vs 19), with 35 % overshoot. If a learned controller looks wrong, check the reward first.**

# Search in log-gains, and respect the stability cliff

- A slice of J over (Kp, Ki) with Kd at its optimum, on log axes. The orange path is the mean of the REINFORCE policy (slide 7) moving from ZN to the optimum.





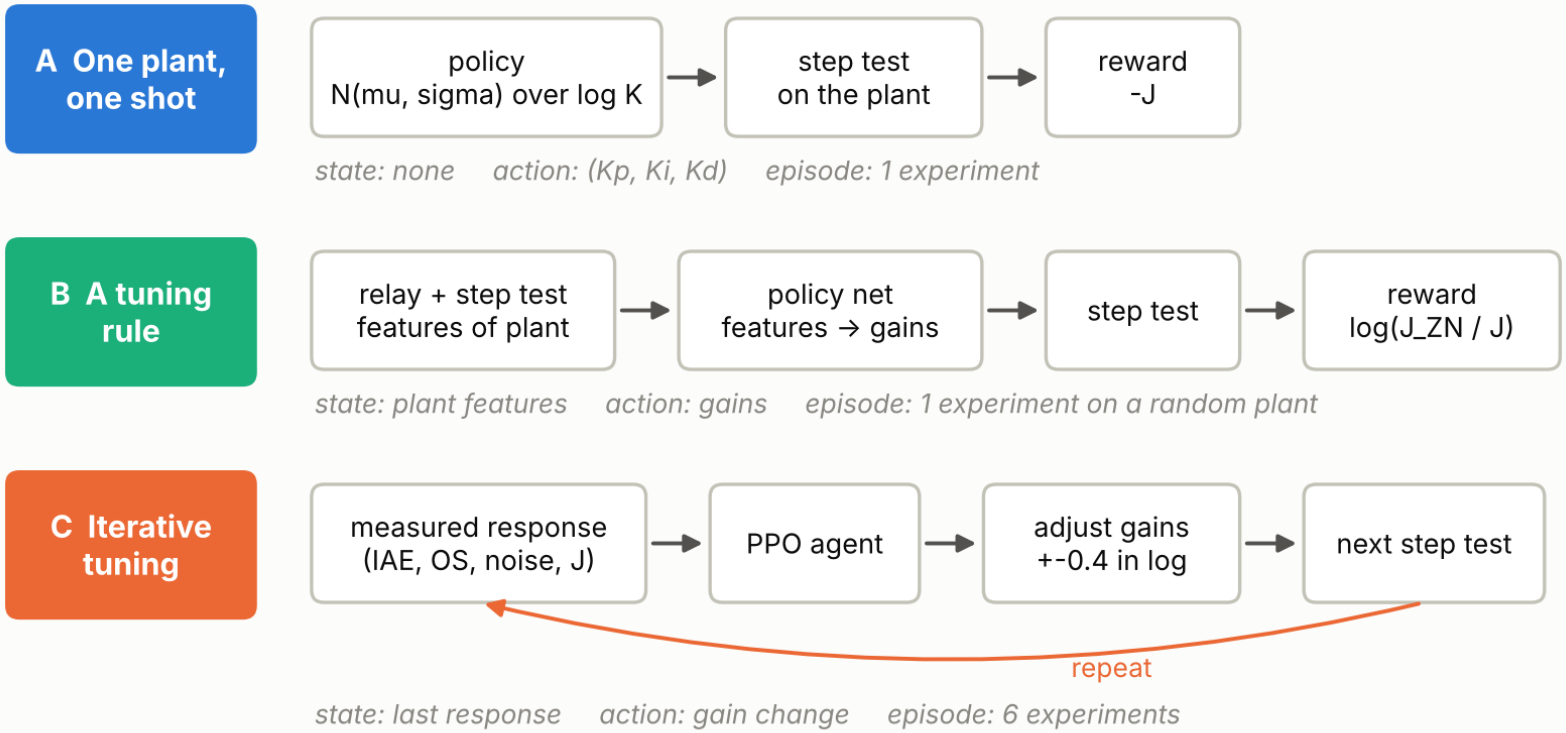


**13 % of this box is unstable, and there J is flat: no gradient points back to safety. ZN classic sits near the edge. Act on log K, and always start from a stable controller.**

FORMULATIONS

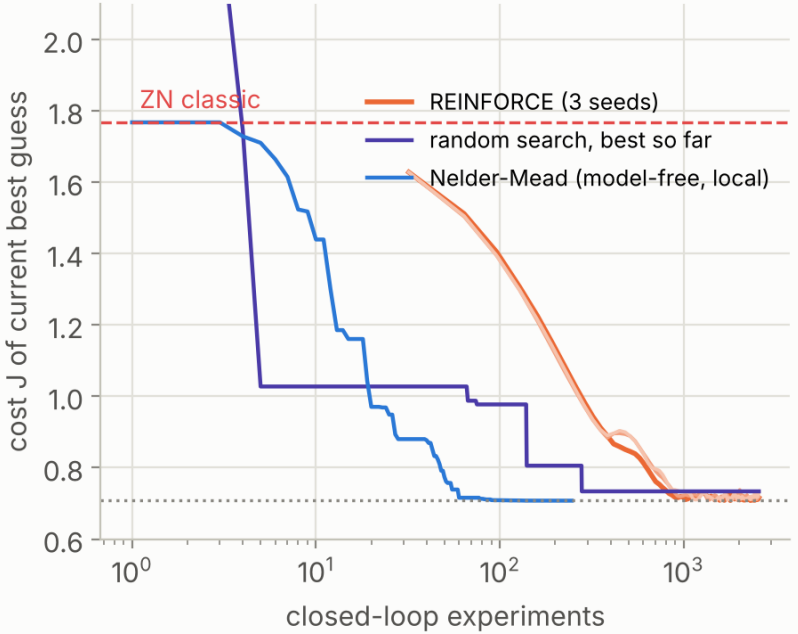
# Three ways to cast PID tuning as reinforcement learning

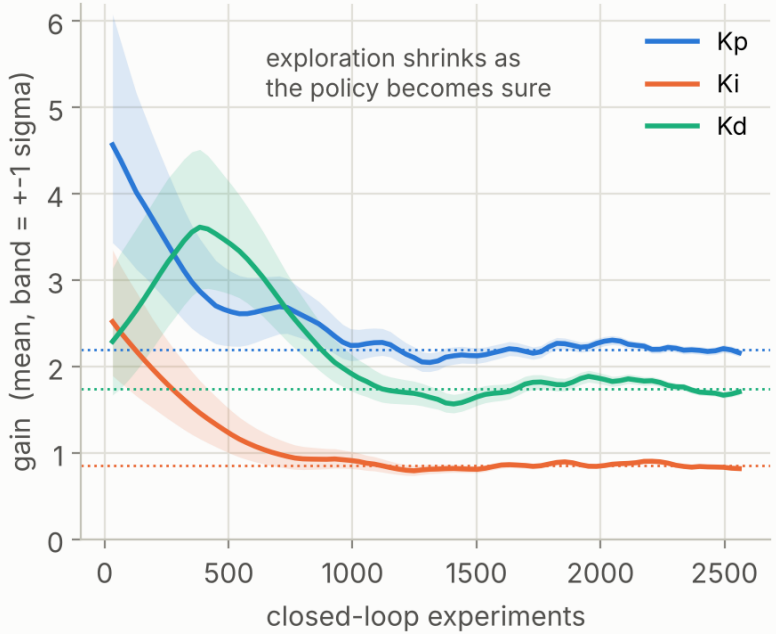
- "Use RL to tune a PID" can mean very different MDPs. Before writing code, fix the state, the action, the reward and what one episode is.




**A and B are contextual bandits, with one decision per episode. C is a genuine multi-step MDP: what the agent tries now changes what it knows next. A fourth family, RL as the controller at 50 Hz, is a different lecture.**

# A — REINFORCE: a policy gradient with six parameters



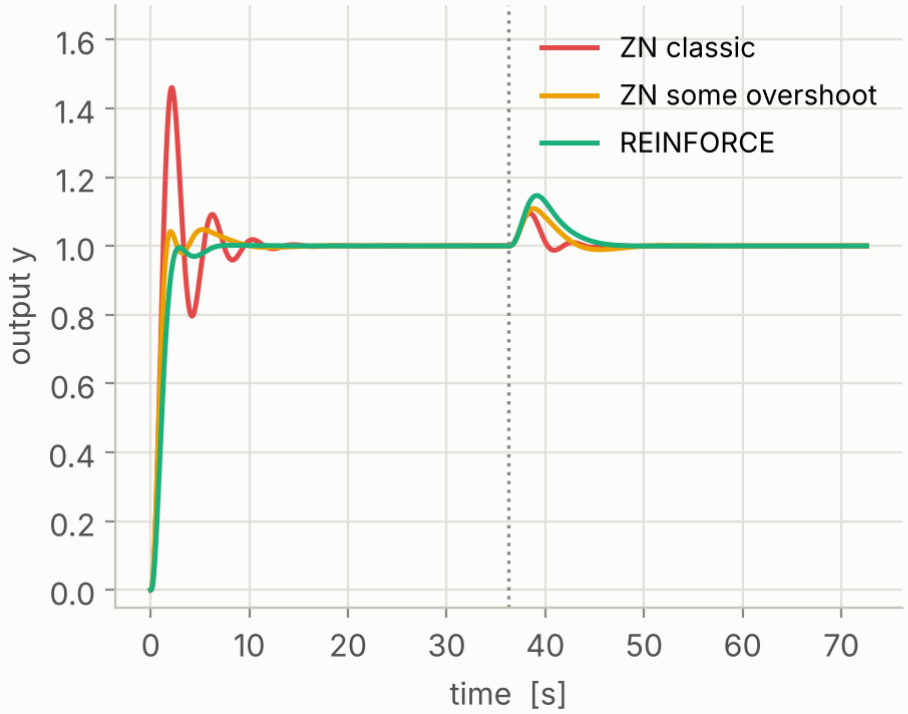

```
eps   = rng.standard_normal((32, 3));   theta = mu + sigma*eps      # 32 sampled log-gain vectors
R     = -cost(exp(theta))                                          # 32 closed-loop step tests
A     = (R - R.mean()) / R.std()                                   # baseline + normalisation
g_mu  = (A[:,None] * eps/sigma).mean(0);  g_logsig = (A[:,None]*(eps**2 - 1)).mean(0)   # -> Adam
```

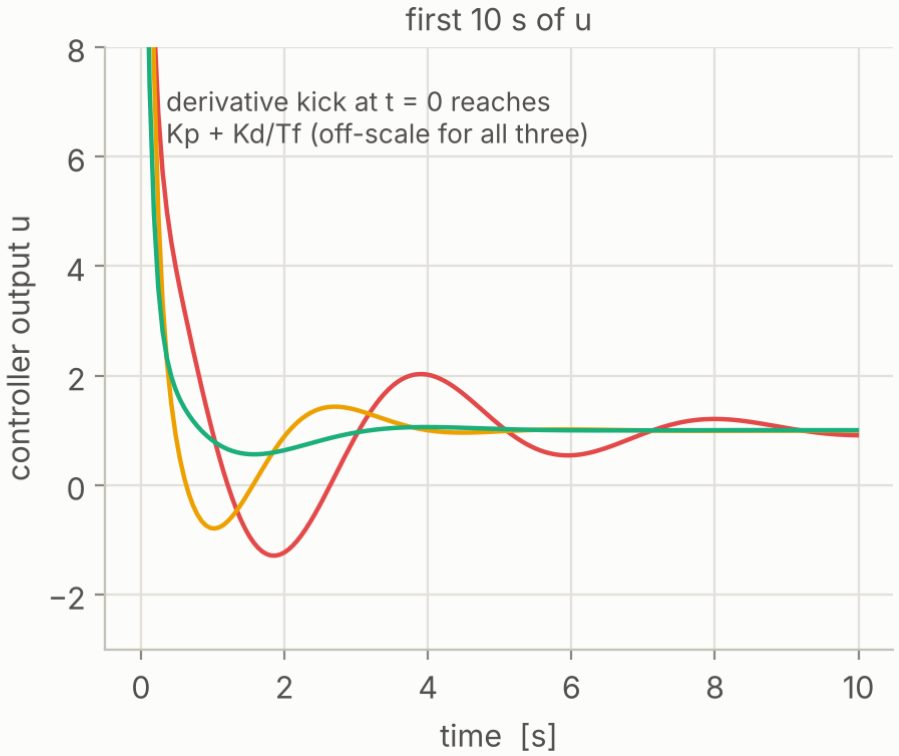
**Policy N(μ, σ²) over θ = log K, started at ZN with σ = 0.3. The score-function gradient needs no derivative of J, which is flat on the unstable plateau.**

RESULT, ONE PLANT

# On the nominal plant RL finds the optimal PID

- REINFORCE ends at Kp = 2.16, Ki = 0.82, Kd = 1.71, within 1.2 % of the exact optimum (Kp 2.20, Ki 0.85, Kd 1.74). All three ZN gains are cut: Kp by 55 %, Ki by 69 %, Kd by 22 %.



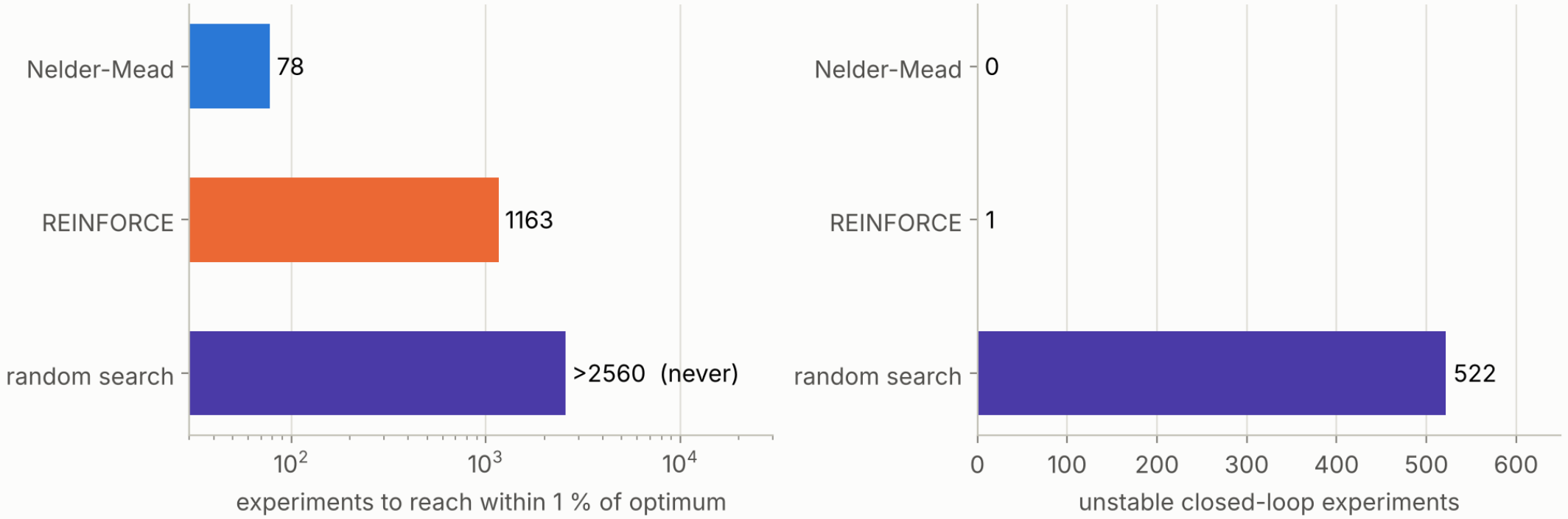




**Overshoot 46 % → 0.2 %, 2 % settling 9.0 s → 5.3 s, phase margin 28° → 70°, Ms 2.43 → 1.37. The price is a slower recovery from the load step.**

THE HONEST COMPARISON

# But on one known plant, a plain optimiser is far cheaper

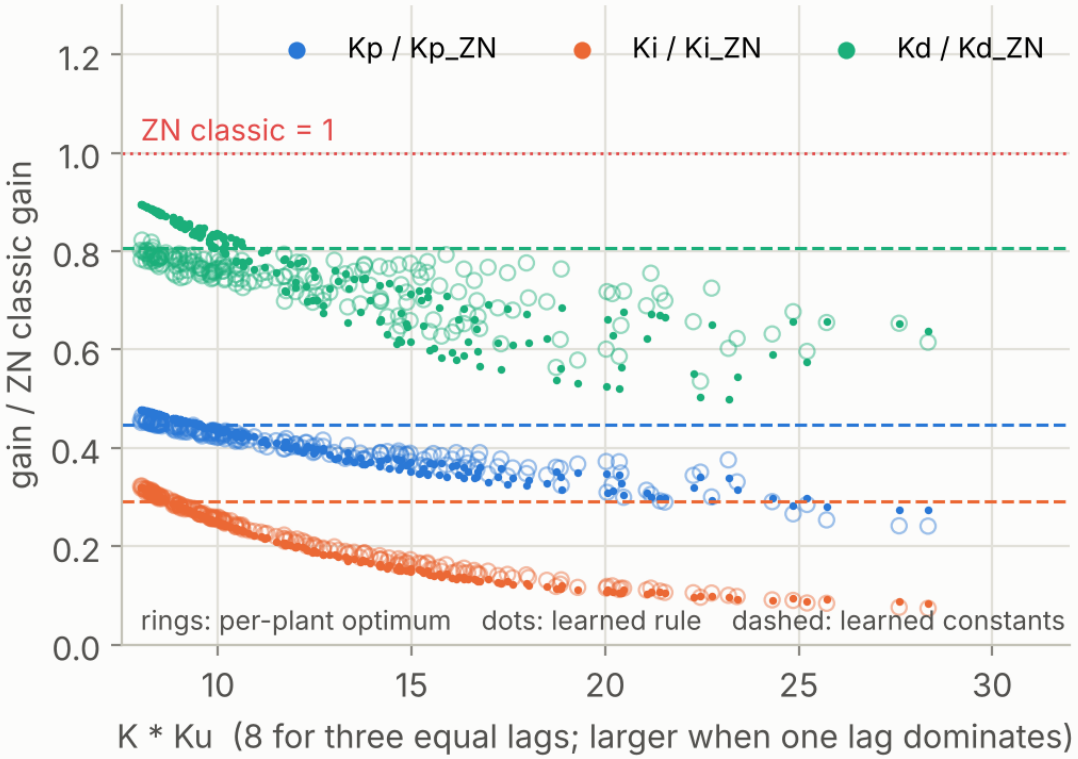
- Same plant, same cost, every method started from ZN classic (random search samples a log-uniform box). Left: experiments needed. Right: how many of them destabilised the loop.

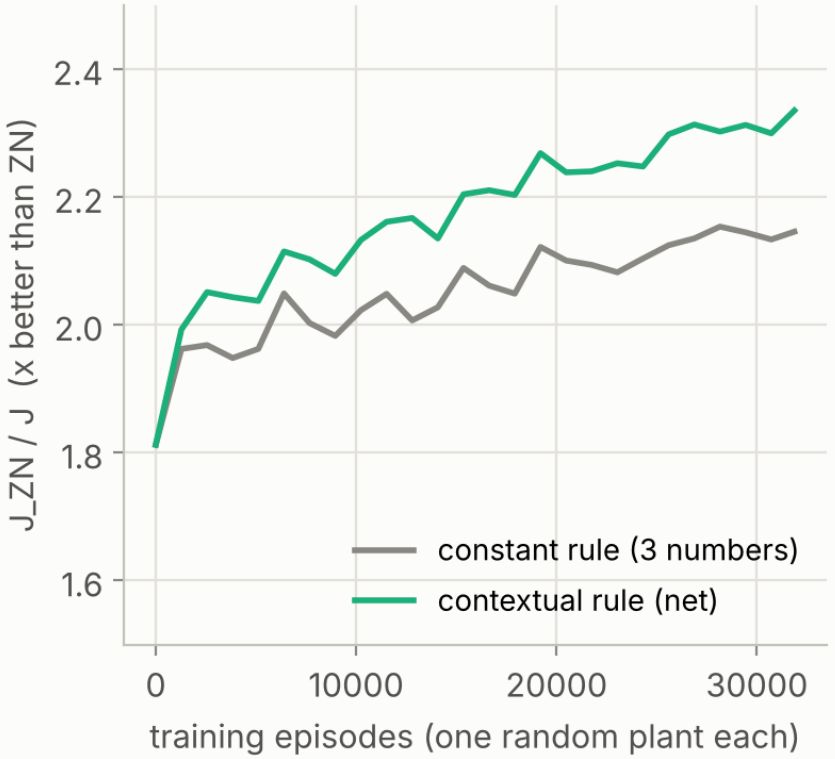



**Nelder–Mead needs 78 experiments; REINFORCE needs about 1163. Its local, stable-start exploration is safe (1 unstable trial against 522 for random search), but one deterministic plant is the wrong place to use RL. Its value is in generalising across plants.**

# B — Learn a better Ziegler–Nichols table

- Train on 4 000 random third-order lags (K in 0.5–2, each τ in 0.25–4 s). State: log(K·Ku) and log(Tu/Tar) from a relay test and a step test. Action: log-multipliers on the ZN gains. Reward: log(J_ZN / J).

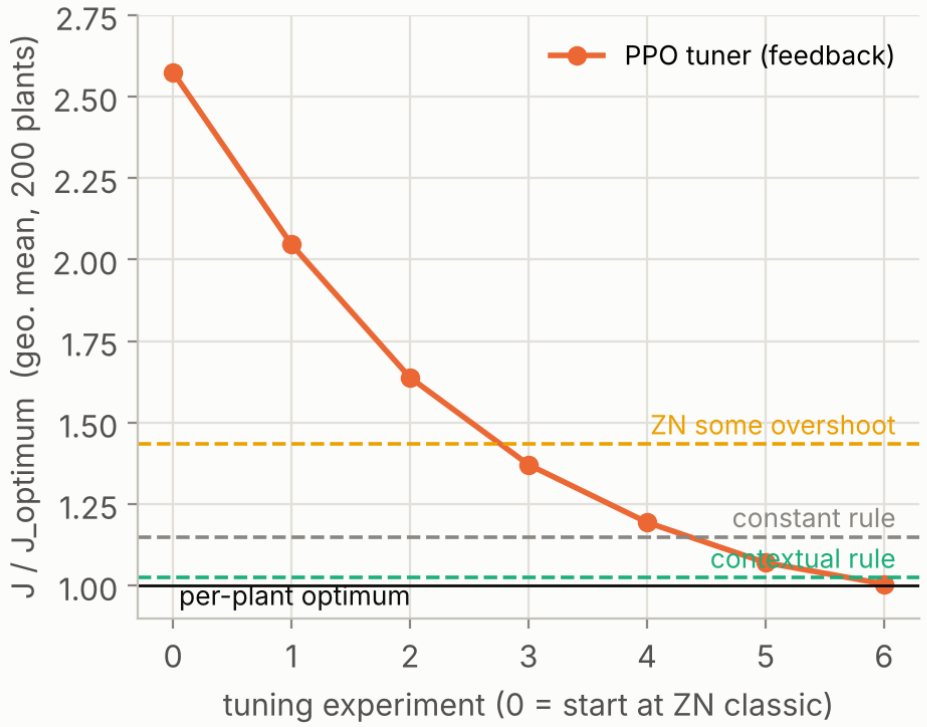


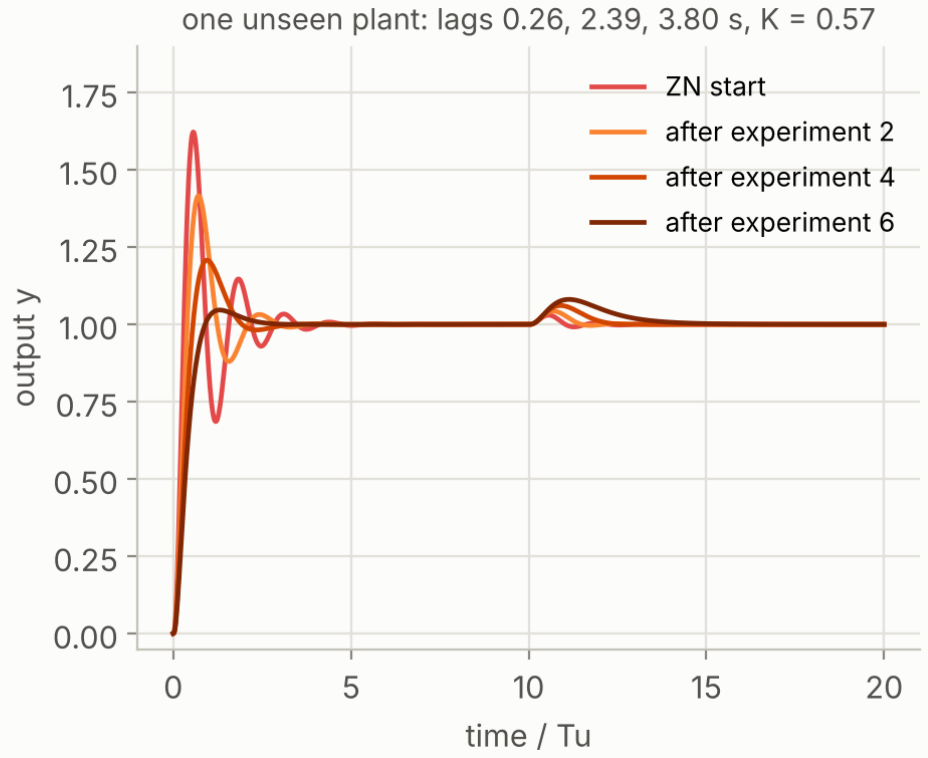




**Three learned constants (×0.45, ×0.29, ×0.81 on ZN) reach 1.15× the per-plant optimum on unseen plants. Letting the rule read the plant brings it to 1.02×.**

# C — An agent that tunes the loop like an engineer

- State: the metrics of the last experiment (IAE, overshoot, noise gain, J, stability), the current gains and the plant features. Action: change each log-multiplier by at most ±0.4. Reward: log(J_prev / J_new). Six experiments, starting at ZN.



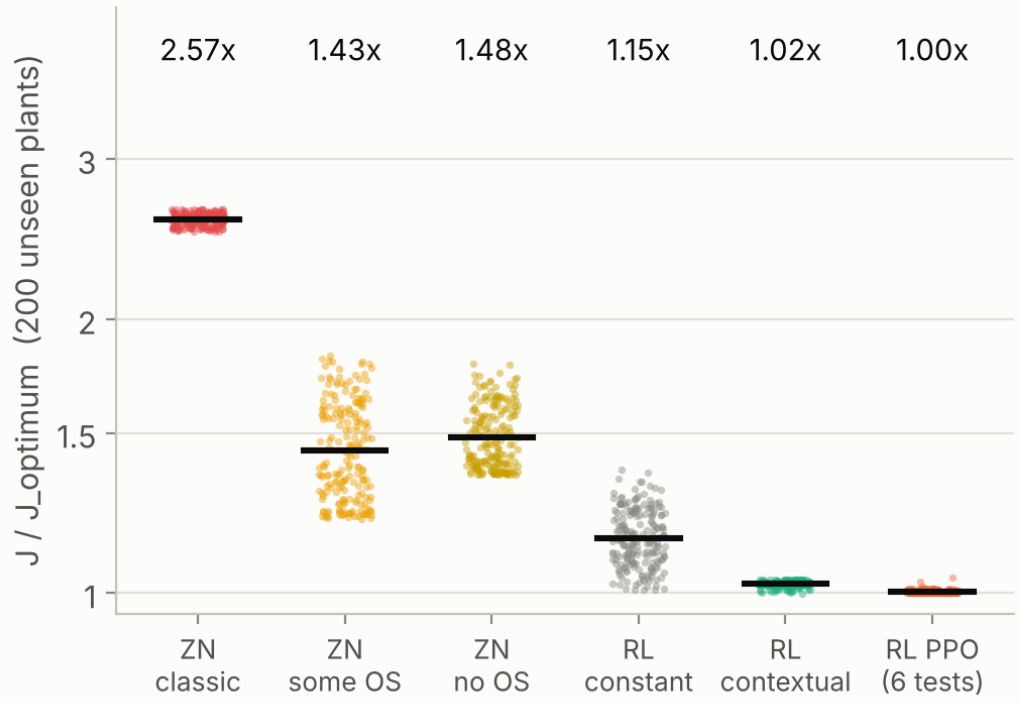



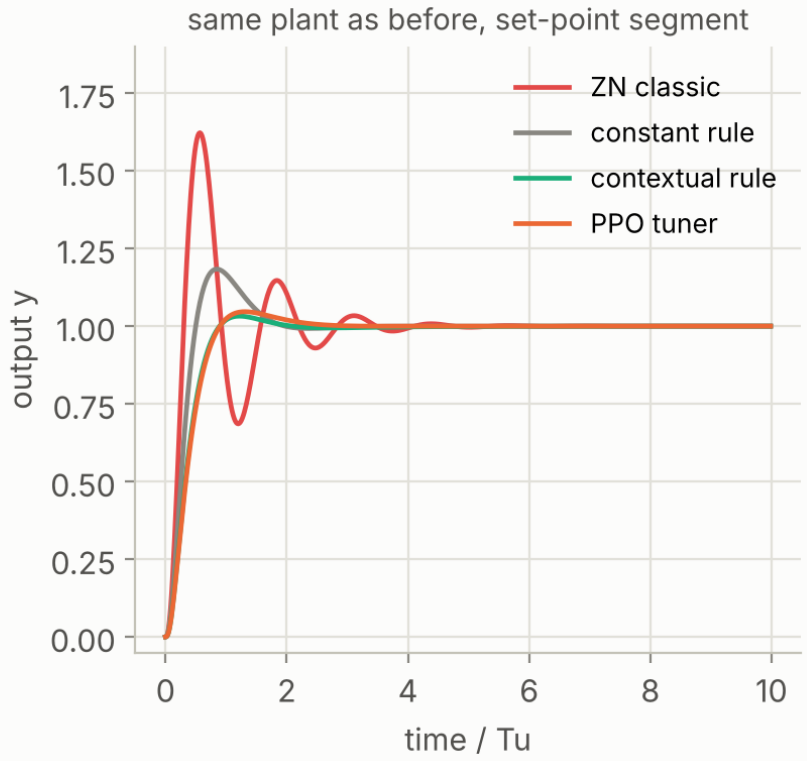

**After 6 experiments the tuner is at 1.004× the per-plant optimum on 200 unseen plants, with 0 unstable experiments. It was trained on 92,160 simulated experiments in 32 s on a CPU.**

HEAD TO HEAD

# Scorecard on 200 plants never seen in training

- Each dot is one plant: its cost divided by the cost of a controller tuned from scratch for that plant (1 440 experiments each). Bars show geometric means.

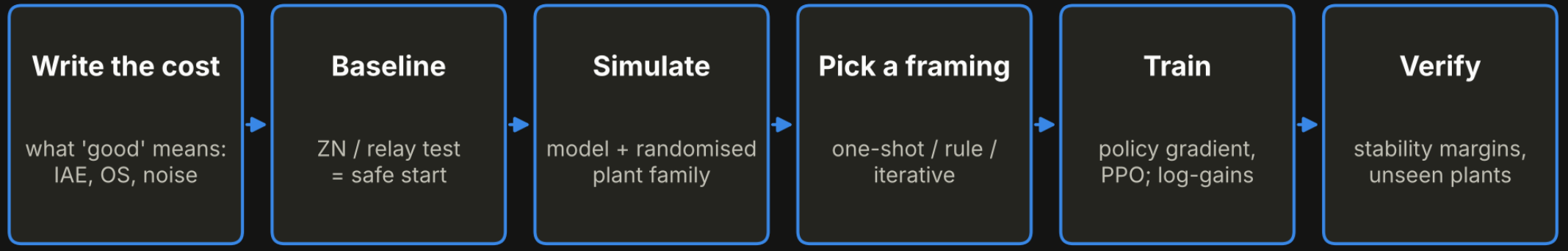



**The ranking is the lesson: a fixed table (2.57×) → learned constants (1.15×) → plant features (1.02×) → closed-loop feedback (1.004×). More information about the plant buys better gains, and RL turns that information into a tuner.**

WHAT TO TAKE AWAY

# RL tunes the PID; the engineer writes the reward and checks the result




1. Write the whole cost: tracking, load rejection, overshoot and noise/actuator effort. Any missing term will be exploited.
2. Start from ZN or another known-stable controller. Act on log-gains, as multipliers relative to that start.
3. Detect instability before the experiment and charge a fixed penalty. Expect a cliff, not a slope.
4. On one known plant use an optimiser (78 vs ~1163 experiments). RL pays off when one training run serves many loops.
5. Make it dimensionless (time in Tu, reward as log J ratio) and feed the features an engineer measures: relay and step tests.
6. Score on unseen plants against a per-plant optimum, not only against ZN.
7. Verify classically before deployment: margins and Ms (ZN classic: PM 28°, Ms 2.43; RL: PM 70°, Ms 1.37).

Generated by Claude and customized by

dew.ninja 2026

Notebook `RL_PID_tuning_third_order_lag.ipynb` · Ziegler & Nichols 1942 · Åström & Hägglund 1984 · Williams 1992 · Schulman et al. 2017 · Lawrence et al. 2022 · Dogru et al. 2022